Be BFFs with the Kubernetes API

Should your frontend talk with kube-apiserver?

Simon Koudijs

Simon Koudijs

`[Software|Product|Cloud|AI] engineer`

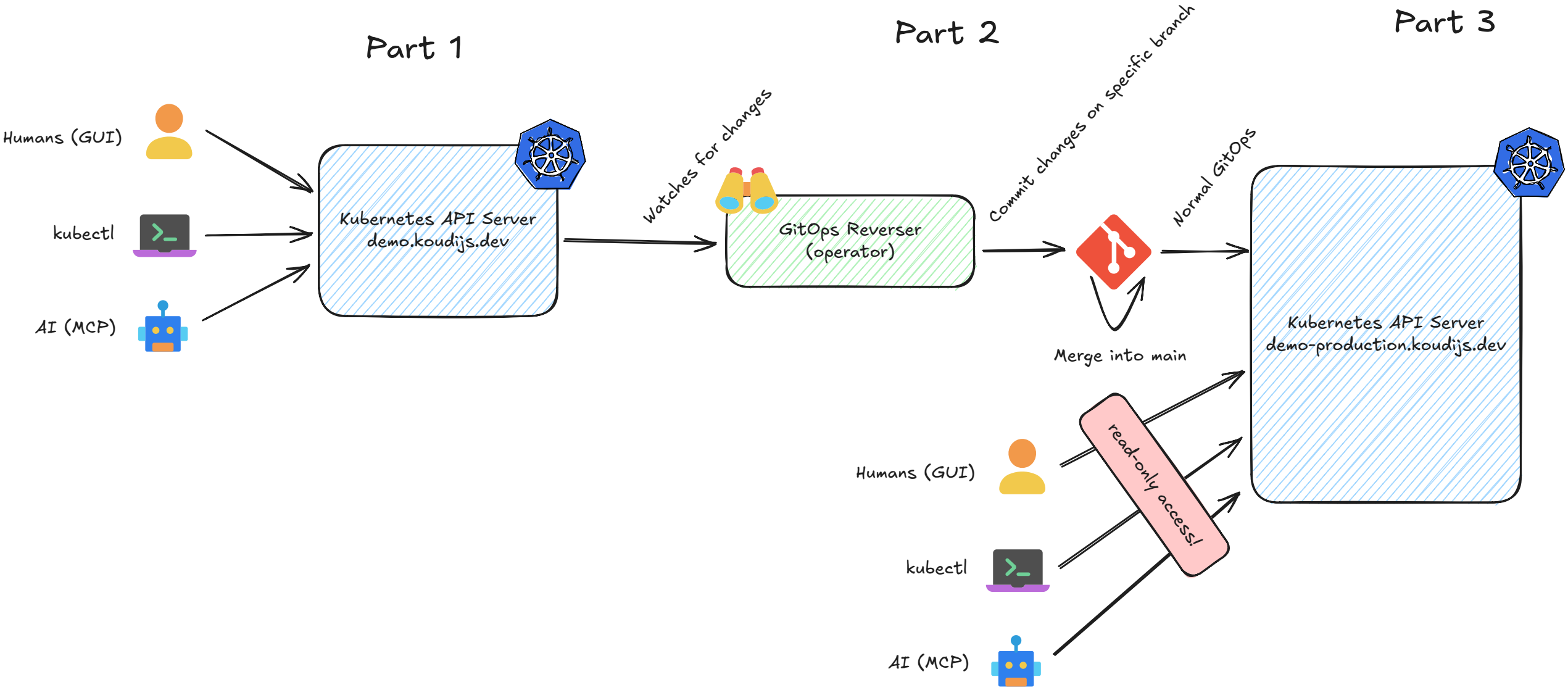
- koudijs.dev: consultancy, training, speaking
- ConfigButler: product to manage your configuration

Groepenindeling Zonnelaan 65
Fase
Groep
Omschrijving
Vaatwasser
CV Ketel + MV
Gashaardmeubel
Combi-magnetr
Voor: Lichtpunt
WCD keuken vo
Kokendwaterkr
Wasdroger
Wasmachine
Achter: woonk
Keuken WCD r
Oven + koelkas
Beltrafo
Garage buiten
Inverter zolder
Korte ha
Iskra SMR
RCM4
RCM4
FORNUISGROEP
AARDLEKSCHAKELAAR "BLAUWE GROEPEN"
AARDLEKSCHAKELAAR "GROENE GROEPEN"
AARDLEKSCHAKELAAR "GELE GROEPEN"
HOOFDSCHAKELAAR
BELTRAFO
LTE-M 450
ISKRA
AM550
Made in EU
2024
Geleverd:
Laag tarief
01428 kWh
IP65
epn

Demo

Grab your phone. Let's log in together.

You make up a name, I don't want your personal details


Part 1
Part 2
Part 3
Humans (GUI)
kubectl
AI (MCP)
Kubernetes API Server
demo.koudijs.dev
Watches for changes
GitOps Reverser
(operator)
Commit changes on specific branch
Merge into main
Normal GitOps
Kubernetes API Server
demo-production.koudijs.dev
read-only access!
Humans (GUI)
kubectl
AI (MCP)

YAML

- You know JSON. You've been sending it to backends for years.
- Kubernetes is famous for YAML.
- Here's the secret: **YAML is just JSON that's afraid of curly braces.**
- The Kubernetes API speaks JSON. YAML is just how humans like to write it down.

```yaml
apiVersion: room-pass.koudijs.dev/v1alpha1
kind: Room
metadata:
  name: demo
  namespace: voter
  resourceVersion: "12910500"
  uid: 6a3240ab-c2c1-4430-b8ac-02e7788228f7
spec:
  allowedReturnURLs:
  - https://demo.koudijs.dev/
  - https://demo-production.koudijs.dev/
  joinCode:
    length: 6
    rotateEvery: 30s
    validFor: 120s
status:
  joinCode:
    code: ZFKDTM
    expiresAt: "2026-10-06T21:36:18Z"
    issuedAt: "2026-10-06T21:34:18Z"
  participantCount: 3
```

Exactly the same font, and data!

```json
{
  "apiVersion": "room-pass.koudijs.dev/v1alpha1",
  "kind": "Room",
  "metadata": {
    "name": "demo",
    "namespace": "voter",
    "resourceVersion": "12910500",
    "uid": "6a3240ab-c2c1-4430-b8ac-02e7788228f7"
  },
  "spec": {
    "allowedReturnURLs": [
      "https://demo.koudijs.dev/",
      "https://demo-production.koudijs.dev/"
    ],
    "attributionNote": "Both are used for demo purposes.",
    "audienceGroup": "demo:voter-audience",
    "endsAt": "2026-12-31T18:00:00Z",
    "enrollment": "Open",
    "joinCode": {
      "length": 6,
      "rotateEvery": "30s",
      "validFor": "120s"
    },
    "title": "Room Pass"
  },
  "status": {
    "joinCode": {
      "code": "ZFKDTM",
      "expiresAt": "2026-10-06T21:36:18Z",
      "issuedAt": "2026-10-06T21:34:18Z"
    },
    "observedGeneration": 2,
    "participantCount": 3
  }
}
```

Type

```
apiVersion: room-pass.koudijs.dev/v1alpha1
kind: Room
```

Group: room-pass.koudijs.dev
Version: v1alpha1
Kind: Room

Generic

```
metadata:
  name: demo
  namespace: voter
  resourceVersion: "12910500"
  uid: 6a3240ab-c2c1-4430-b8ac-02e7788228f7
```

Spec

```
spec:
  allowedReturnURLs:
  - https://demo.koudijs.dev/
  - https://demo-production.koudijs.dev/
  joinCode:
    length: 6
    rotateEvery: 30s
    validFor: 120s
```

Desired State
(your settings)

Status

```
status:
  joinCode:
    code: ZFKDTM
    expiresAt: "2026-10-06T21:36:18Z"
    issuedAt: "2026-10-06T21:34:18Z"
  participantCount: 3
```

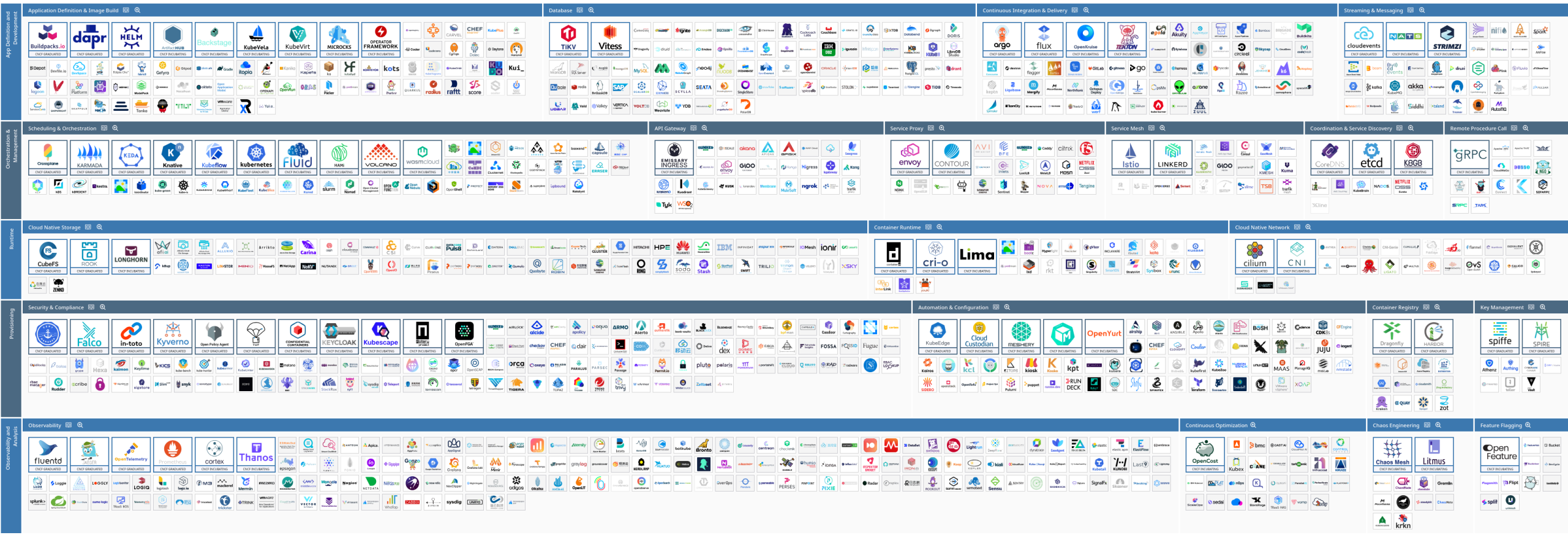
Current State
Can change often
(from your operator)

(removed some fields for clarity)

A quick word on Kubernetes

- Turned **10** last year.
- KubeCon, the yearly CNCF conference: about **13,000** people.
- Known for running containers, at a very large scale.
- But that's not why we're here today.

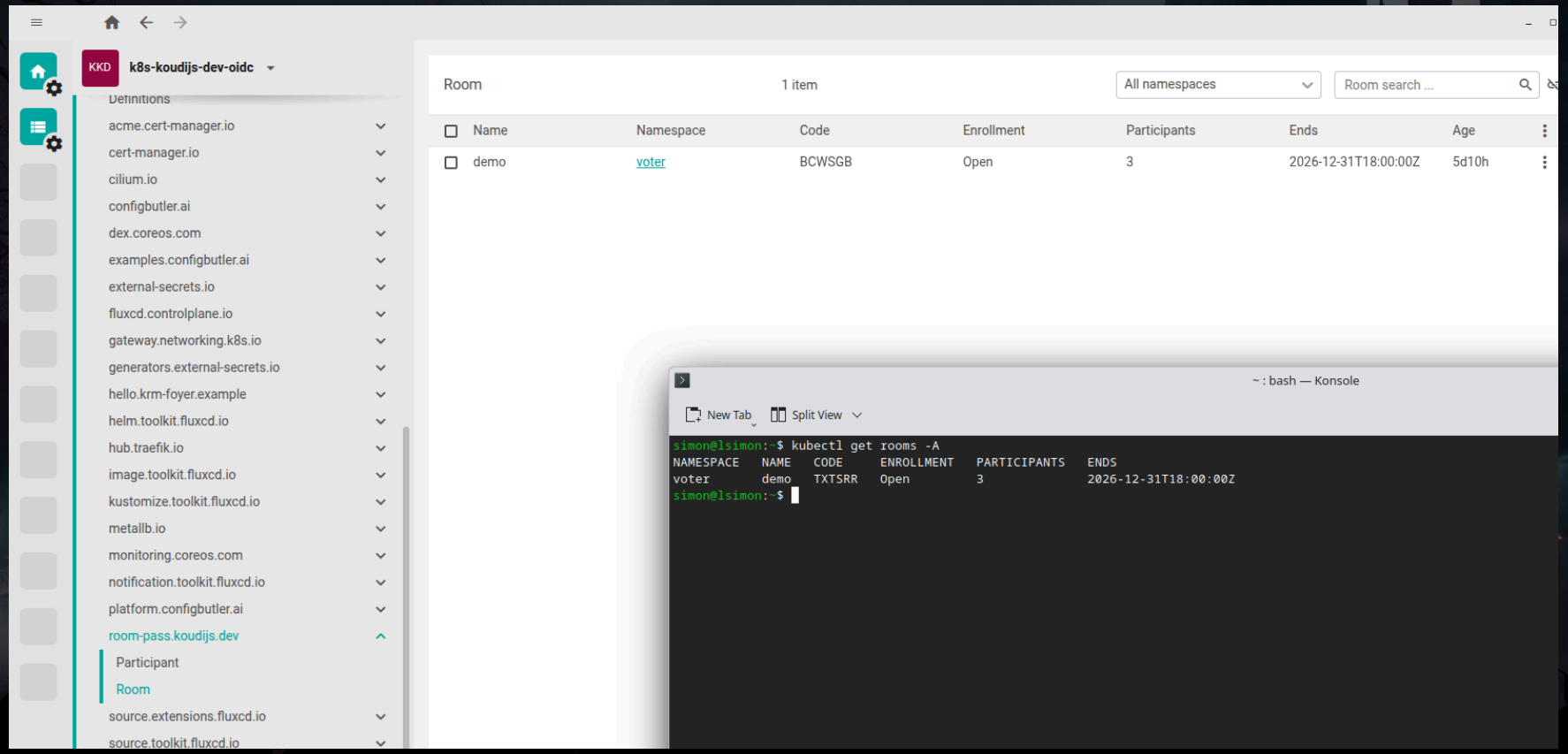
FRONT
MANIA!
CONFERENCE
2026
AMSTERDAM
DAF
DAF
Heineken
LAVAZZA


App Definition and Development
Application Definition & Image Build
Buildpacks.io
dapr
HELM
ArtifactHUB
Backstage
KubeVela
KubeVirt
MICROCKS
OPERATOR FRAMEWORK
Database
TiKV
Vitess
Continuous Integration & Delivery
argo
flux
OpenKruise
TEKTON
Streaming & Messaging
cloudevents
NATS
STRIMZI
Orchestration & Management
Scheduling & Orchestration
Crossplane
KARMADA
KEDA
Knative
Kubeflow
kubernetes
FLUID
HAMi
VOLCANO
wasmcloud
API Gateway
EMISSARY INGRESS
Service Proxy
envoy
CONTOUR
Service Mesh
Istio
LINKERD
Coordination & Service Discovery
CoreDNS
etcd
K8GB
Remote Procedure Call
gRPC
Runtime
Cloud Native Storage
CubeFS
ROOK
LONGHORN
Container Runtime
cri-o
Lima
Cloud Native Network
cilium
CNI
Provisioning
Security & Compliance
Falco
in-toto
Kyverno
Open Policy Agent
KEYCLOAK
Kubescape
OpenFGA
Automation & Configuration
KubeEdge
Cloud Custodian
MESHERY
OpenYurt
Container Registry
Dragonfly
HARBOR
Key Management
spiffe
SPIRE
Observability and Analysis
Observability
fluentd
OpenTelemetry
Prometheus
cortex
Thanos
Continuous Optimization
OpenCost
Chaos Engineering
Chaos Mesh
Litmus
Feature Flagging
OpenFeature

We almost have as many operators as you have frontend frameworks

- Almost all of them come down to the same thing:
  a set of API resources that describe the desired state of something.
- Most of it is about infrastructure. Is that a bummer?
- Not really: the API underneath is one of the best I've seen.
- So many REST discussions later, and it was there all along.

KUBECTL


KKD
k8s-koudijs-dev-oidc
Definitions
acme.cert-manager.io
cert-manager.io
cilium.io
configbutler.ai
dex.coreos.com
examples.configbutler.ai
external-secrets.io
fluxcd.controlplane.io
gateway.networking.k8s.io
generators.external-secrets.io
hello.krm-foyer.example
helm.toolkit.fluxcd.io
hub.traefik.io
image.toolkit.fluxcd.io
kustomize.toolkit.fluxcd.io
metallb.io
monitoring.coreos.com
notification.toolkit.fluxcd.io
platform.configbutler.ai
room-pass.koudijs.dev
Participant
Room
source.extensions.fluxcd.io
source.toolkit.fluxcd.io
Room
1 item
All namespaces
Room search ...
Name
Namespace
Code
Enrollment
Participants
Ends
Age
demo
voter
BCWSGB
Open
3
2026-12-31T18:00:00Z
5d10h
~ : bash — Konsole
New Tab
Split View
simon@lsimon:~$ kubectl get rooms -A
NAMESPACE NAME CODE ENROLLMENT PARTICIPANTS ENDS
voter demo TXTSRR Open 3 2026-12-31T18:00:00Z
simon@lsimon:~$

KRM: the Kubernetes Resource Model

- Everything is a **resource**: a document with a type, a name and fields.
- Kubernetes ships a few hundred types out of the box.
- **You can add your own.** Model *your* application, in *your* terms.
- Each new type immediately gets the whole pipeline: auth, RBAC, validation, admission, history and live updates.

Define your own resource

```yaml
apiVersion: voter.example.com/v1
kind: Question
metadata:
  name: first-question
spec:
  text: "Have you ever typed kubectl?"
  options: ["yes", "no", "what is kubectl?"]
status:
  votes: { yes: 12, no: 30, "what is kubectl?": 8 }
```

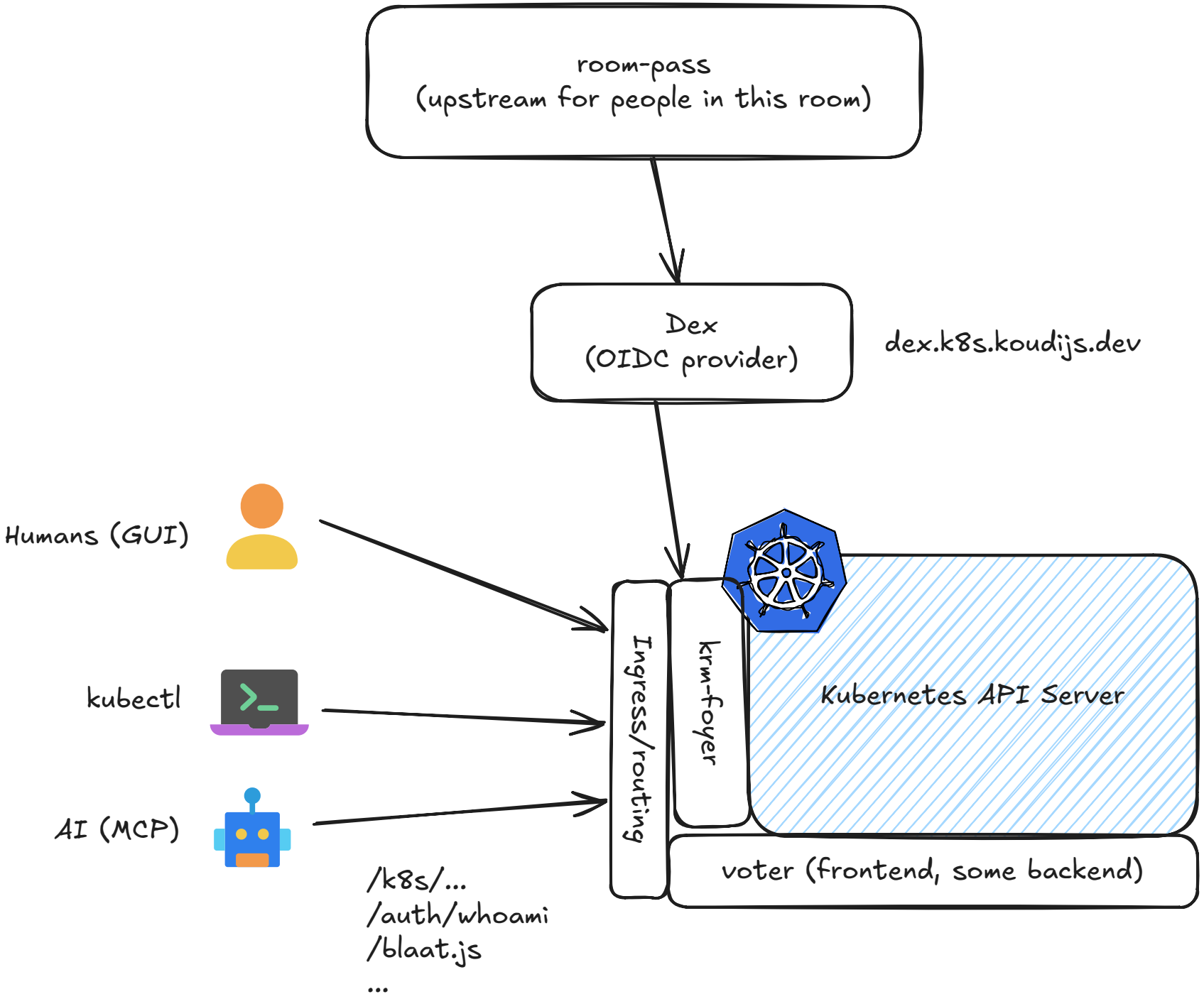
- **spec:** what you want (written by the frontend).
- **status:** what actually happened (written by the backend).

OIDC

- You just logged in. What actually happened?
- **OAuth 2.0:** how an app *gets a token* on your behalf.
- **OIDC:** OAuth plus *who you are*, in a signed ID token (a JWT).
- The API server trusts the issuer (**Dex**) and reads your name and groups from the token.
- Dex asks **room-pass** who you are.
- Your browser never holds that token: just a **server-side cookie**.

How the voter works: krm-foyer

- Your browser never sees a Kubernetes token.
- **krm-foyer** sits on the same origin as the app:
  - login (OIDC via room-pass), **server-side cookie only**
  - forwards each request with *your* identity
  - leaves every access decision to the API server
- The app itself is a normal frontend that uses `fetch`.

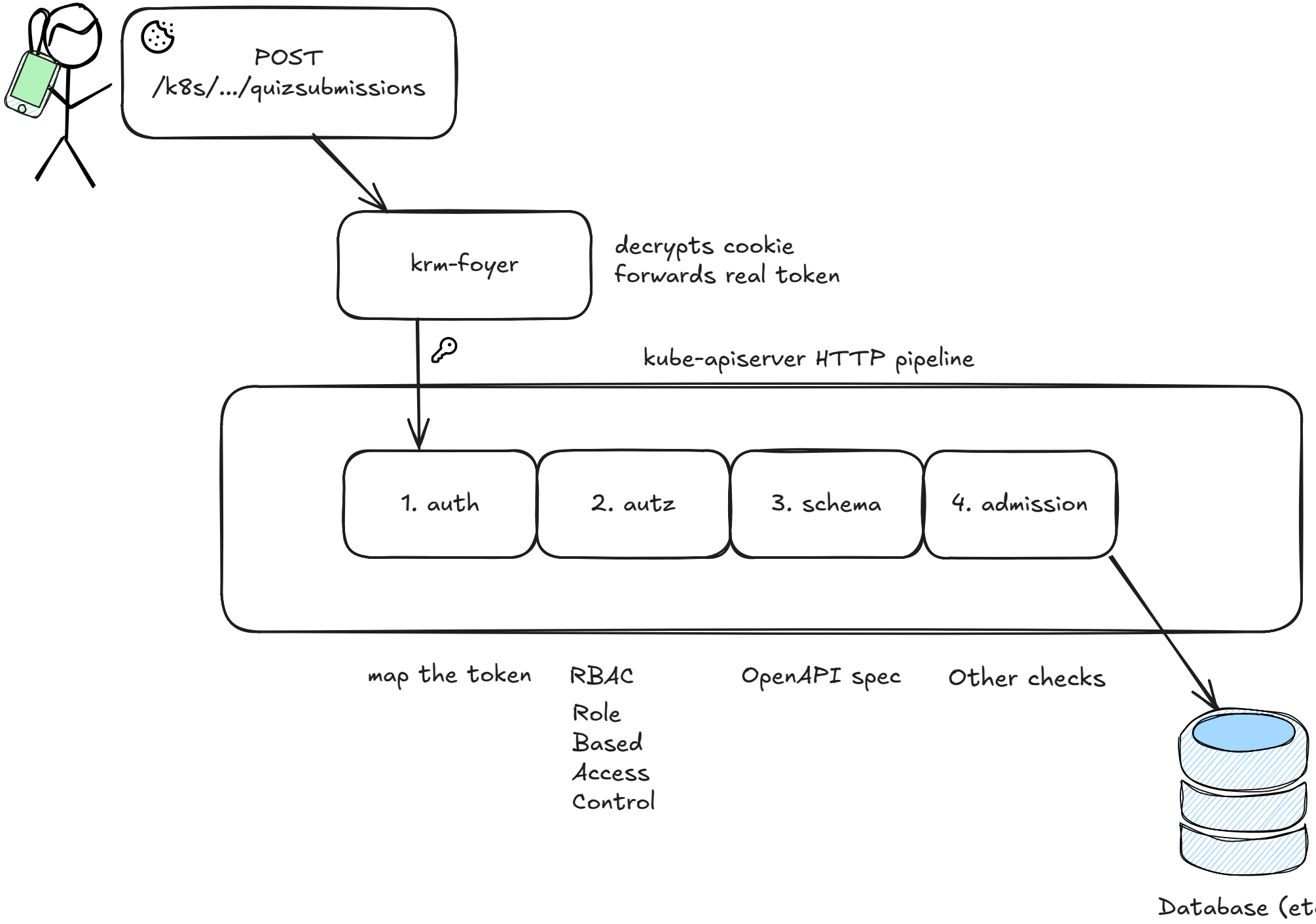

room-pass
(upstream for people in this room)
Dex
(OIDC provider)
dex.k8s.koudijs.dev
Humans (GUI)
kubectl
AI (MCP)
Ingress/routing
krm-foyer
Kubernetes API Server
voter (frontend, some backend)
/k8s/...
/auth/whoami
/blaat.js
...

What every serious API has to do

Every request needs answers to the same questions:

1. **Authentication:** who are you?
2. **Authorization:** are you allowed to do this?
3. **Schema validation:** does this even make sense?
4. **Admission:** are there other rules this change has to pass?

Only then does anything get stored.


POST
/k8s/.../quizsubmissions
krm-foyer
decrypts cookie
forwards real token
kube-apiserver HTTP pipeline
1. auth
2. autz
3. schema
4. admission
map the token
RBAC
Role
Based
Access
Control
OpenAPI spec
Other checks
Database (etcd)

This is not about your production infra

- No pods, no nodes, no containers, no scheduling.
- **A small, dedicated Kubernetes API server as the backend of your product.**
- Tools like **kcp** or **Capsule** make that a light, isolated setup.
- Every component you'll see today is **open source**.

Demo

First question

Answer on your phone.

What just happened?

- Some of you could vote. Some of you got a **403 Forbidden**.
- Nobody changed the app.
- I changed **who is allowed to do what**.
- That's step 2 of the pipeline: **RBAC**.

RBAC: Role Based Access Control

```yaml
kind: Role
metadata:
  name: voter-audience-ballot
  namespace: voter
rules:
  - apiGroups: ["examples.configbutler.ai"]
    resources: ["quizsubmissions"]
    verbs: ["create"]
---
kind: RoleBinding
metadata:
  name: voter-audience-ballot-frontend-svelte
  namespace: voter
subjects:
  - kind: Group
    name: demo:frontend-svelte
roleRef:
  kind: Role
  name: voter-audience-ballot
```

Admission: checks on the data

Before anything is stored: does this particular change make sense?

```
validations:
  - expression: "object.spec.price <= 10.0"
    message: "Nobody pays more than €10 for a coffee"
```

- A `ValidatingAdmissionPolicy` in CEL: no server needed.
- Or a **webhook**: the API server calls *your* HTTP endpoint and asks "is this OK?"

Live updates: watch

- Kubernetes can **watch**: open a stream, and every change comes in as it happens.
- krm-foyer serves that to the browser as good old **Server-Sent Events**.
- **krm-stream** keeps the latest version in every browser and shares streams, so 200 phones don't overwhelm the API server.

Open source, for frontenders

`npm i @configbutler/krm-stream`: zero runtime dependencies, no UI framework

- **SSE helpers:** a Kubernetes watch becomes a plain `EventSource`. Ten tabs, one upstream watch.
- **Three-way merge:** a live update is merged into what you are typing, instead of clobbering it.
- It knows Kubernetes: `resourceVersion`, lists keyed by `name`, redacted fields never written back.

When two people change the same thing

- Every resource has a `resourceVersion`.
- You send the version you based your change on.
- Someone else was faster? **409 Conflict.**
- Think `ETag` and `If-Match`, but built in for every type.

Three-way merge

- **base:** the version you started editing (`resourceVersion` 41)
- **mine:** your local draft
- **theirs:** the live version that just came in through the watch (42)

Someone changed **another field**? Merged. You keep typing.
Someone changed **the same field**? A real conflict: shown to you, never silently overwritten.

> Git does exactly this when you merge a branch.

Now for real: CoffeeConfig

- The voter was a warm-up. Now let's configure a coffee shop.
- Menu, prices, sizes, opening hours: **configuration**.
- Every setting is a resource. Every screen is a view on those resources.

Not everything belongs in etcd

In the Kubernetes API	Somewhere else
Menu and prices	Coffee orders
Opening hours	Payments
Feature flags	Logs and events
Who may change what	Anything high-frequency

- etcd is no relational database: no joins, weak queries, objects capped around 1.5 MB.
- **Configuration**: changes rarely, matters a lot, often edited together. That's the sweet spot.

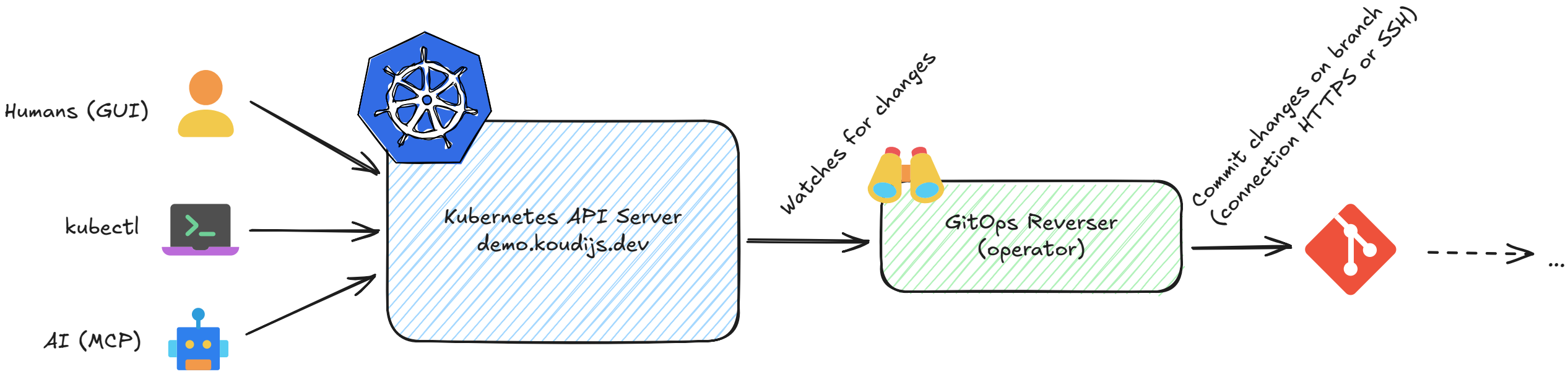
Demo

You are all admin. Configure my coffee shop.

I'm editing too. Let's see whose change wins.

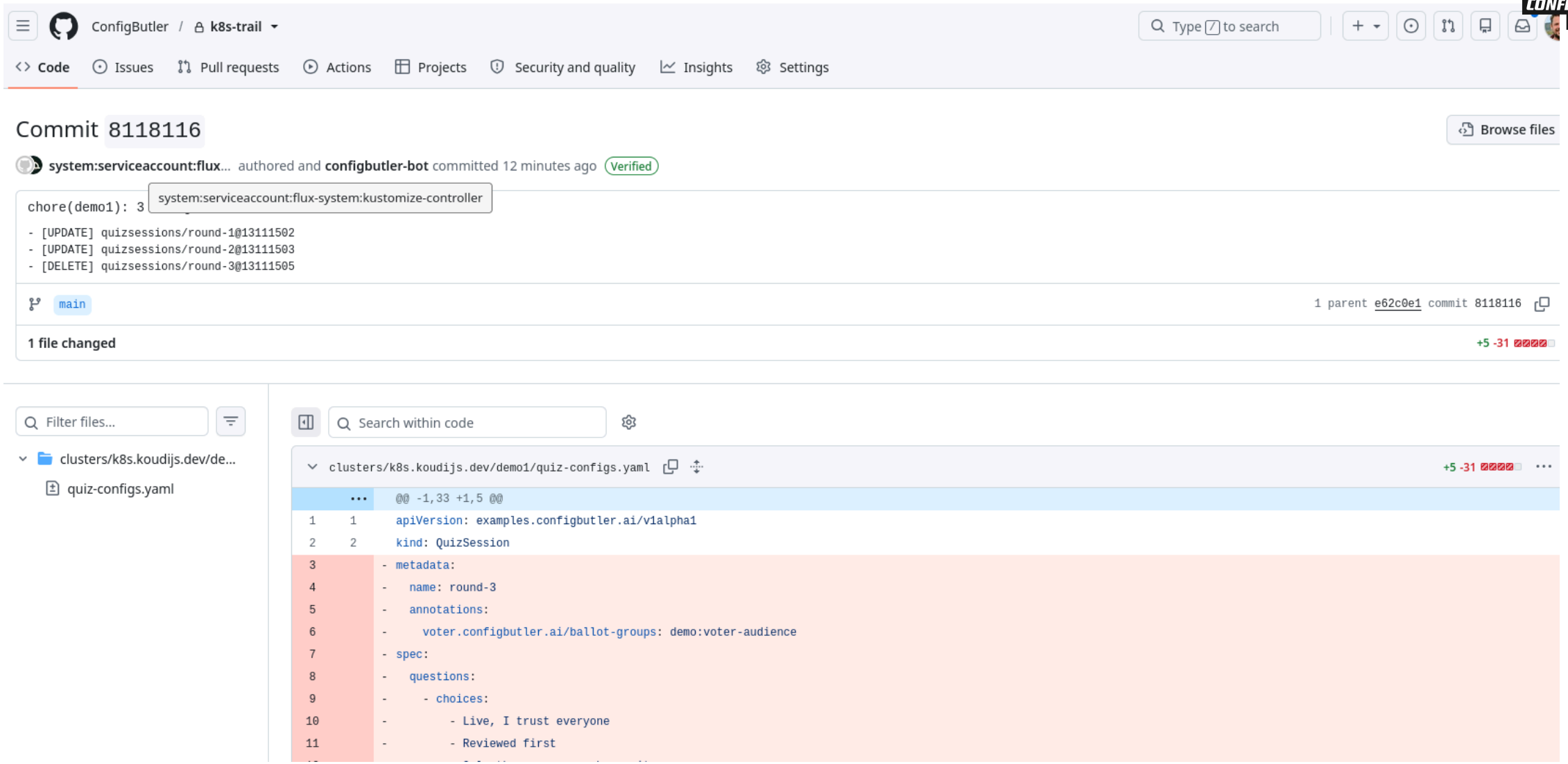
GitOps

- **Git is the source of truth.** A robot makes the servers match it.
- Every change: a commit, a review, a rollback if needed.
- Today we start **in reverse**: from the API, back into git.


Humans (GUI)
kubectl
AI (MCP)
Kubernetes API Server
demo.koudijs.dev
Watches for changes
GitOps Reverser
(operator)
Commit changes on branch
(connection HTTPS or SSH)
...

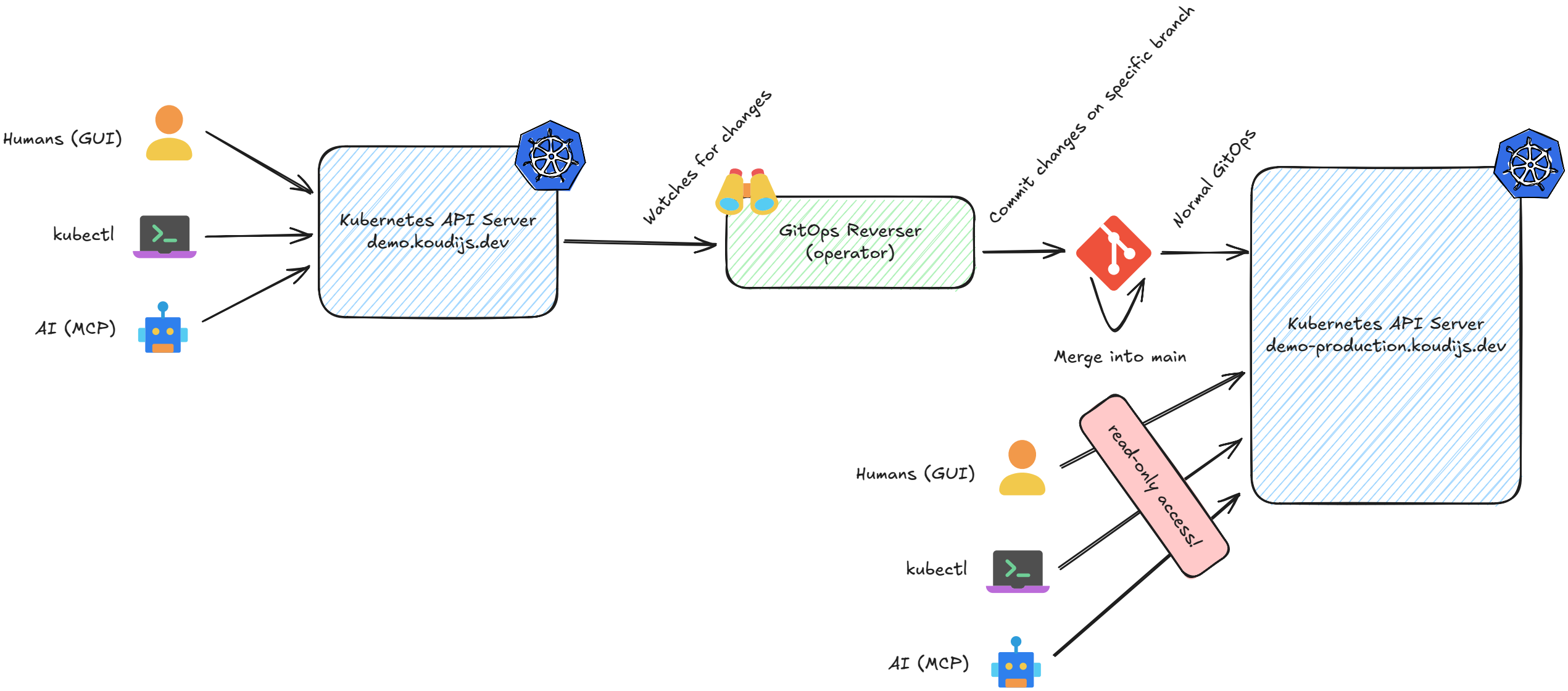
Every change is a document

- Every change you just made is a YAML document with an **author**.
- So we can stream them into **git**.
- Your name, your change, your timestamp.


ConfigButler / k8s-trail
Type / to search
Code
Issues
Pull requests
Actions
Projects
Security and quality
Insights
Settings
Commit 8118116
Browse files
system:serviceaccount:flux... authored and configbutler-bot committed 12 minutes ago
Verified
chore(demo1): 3
system:serviceaccount:flux-system:kustomize-controller
- [UPDATE] quizsessions/round-1@13111502
- [UPDATE] quizsessions/round-2@13111503
- [DELETE] quizsessions/round-3@13111505
main
1 parent e62c0e1 commit 8118116
1 file changed
+5 -31
Filter files...
clusters/k8s.koudijs.dev/de...
quiz-configs.yaml
Search within code
clusters/k8s.koudijs.dev/demo1/quiz-configs.yaml
+5 -31
@@ -1,33 +1,5 @@
1 1 apiVersion: examples.configbutler.ai/v1alpha1
2 2 kind: QuizSession
3 - metadata:
4 - name: round-3
5 - annotations:
6 - voter.configbutler.ai/ballot-groups: demo:voter-audience
7 - spec:
8 - questions:
9 - - choices:
10 - - Live, I trust everyone
11 - - Reviewed first

And then: promote to production

- You just changed **demo.koudijs.dev**: that's the *test* environment.
- **demo-production.koudijs.dev** has **no edit button**.
- Your changes become a pull request: review, merge to `main`, and a GitOps tool rolls them out.
- Slower than a live edit? Yes. **That's the point** for the risky settings.


Humans (GUI)
kubectl
AI (MCP)
Kubernetes API Server
demo.koudijs.dev
Watches for changes
GitOps Reverser
(operator)
Commit changes on specific branch
Merge into main
Normal GitOps
Kubernetes API Server
demo-production.koudijs.dev
read-only access!
Humans (GUI)
kubectl
AI (MCP)

The weird and dangerous parts, honestly

- **etcd is not your database.** Configuration yes, transactions no.
- **Browser → API server** needs care: OIDC, a proxy, a dedicated API server.
- **Someone has to run it**, and someone needs Kubernetes knowledge.
- **It's on the edge.** A lot is not there yet; I had to build parts myself.
- Why do it anyway? High-quality open source, modern standards, and an API model that has been tested at huge scale.

When (not) to use it

A good fit

- Your company already runs Kubernetes, or has a platform team.
- Configuration-heavy products, self-service portals.
- Backenders and architects are on board, not only the frontend team.

Not a good fit

- Transactional or high-frequency data.
- No Kubernetes skills or support in the team.

You don't have to replace every API. **Start small.**

All open source: run it yourself

- **room-pass**: OIDC for everyone, without a privacy problem
  github.com/sunib/room-pass
- **krm-foyer**: browser login and live resources on one origin
  github.com/ConfigButler/krm-foyer
- **krm-stream**: frontend lib keeps every browser in sync without overwhelming the API server
  github.com/ConfigButler/krm-stream
- **gitops-reverser**: streams every change back into git
  github.com/ConfigButler/gitops-reverser

Weird, brilliant, or dangerous?

WEIRD

Yes. A browser talking to kube-apiserver is not what Kubernetes was built for.

DANGEROUS

Only on your production cluster. A dedicated API server gives you auth, RBAC and admission for free.

BRILLIANT

For configuration: live, attributed, reviewed in git. Start small.

Kubernetes may not become your best friend forever. **But it could be your Backend for Frontend.**

Thank you! · **#FrontMania**

configbutler.ai

koudijs.dev/speaking